

Attack Scenarios and Embedded Intrusion Detection for Space Systems

1st Louis Lolive

IRT Saint Exupéry / LAAS-CNRS

Toulouse, France

louis.lolive@laas.fr

2nd Guillaume Auriol

LAAS-CNRS

Univ. Toulouse, INSA, France

guillaume.auriol@laas.fr

3rd Vincent Nicomette

LAAS-CNRS

Univ. Toulouse, INSA, France

vincent.nicomette@laas.fr

4th Florent Galtier

LAAS-CNRS

Toulouse, France

florent.galtier@laas.fr

5th Simone Urbano

Starion Group France

Toulouse, France

s.urbano@stariongroup.eu

6th Jacques Girard

Thales Alenia Space

Toulouse, France

jacques.girard@thalesaleniaspace.com

Abstract—Space systems have become strategic targets for cybersecurity attacks. It is thus crucial to address the security of such systems, in order to detect and mitigate these attacks. For that purpose, this paper proposes the design of an Intrusion Detection System (IDS) embedded on a satellite, aiming at detecting a large range of cyberattacks, thanks to the deployment of several probes and detection strategies. This IDS is designed to take into account the specific characteristics of space systems, including their limited resources, particular architectures, the nature of the data they process, and the communication protocols they use. The paper also describes a proof of concept of this IDS that has been implemented within the NASA's NOS³ platform, as well as a set of attacks that we designed and tested to assess the relevance of our IDS. The impact on performance of the proposed IDS is also assessed.

Index Terms—cybersecurity, intrusion detection, space systems, NOS³

I. INTRODUCTION

Space systems are increasingly essential for communication, navigation, and national defense. Historically, they have been considered protected only by their complexity and limited accessibility, with a small number of companies involved in the space development field. Nowadays, the NewSpace sector and the use of COTS (especially in nanosatellites) have completely changed the way satellites are built and operated. Recent incidents, such as the VIASAT attack [1], the OPS-SAT hack at CYSAT [2], and the Hack-A-Sat capture of Moonlighter [3], have shown that the assumptions regarding security issues are obsolete. In response, research is focusing on enhancing space system security through numerous methods like security by design [4] and advanced intrusion detection systems [5].

The security of space systems must be addressed at various strategic points: ground systems, communication links, and the embedded systems within the satellite itself. While numerous security mechanisms exist to protect on-ground information systems, there is currently less research focused on embedding security directly within the satellite. Our research aims to

fill this gap by exploring Intrusion Detection Systems (IDS) embedded in satellites.

Designing an Intrusion Detection System (IDS) for satellites involves unique challenges due to their resource constraints, limited communication windows, and lack of onboard human intervention. While sharing some features with IoT and automotive systems, satellites require tailored IDS designs to handle domain-specific data like telecommands and telemetry. Key considerations include system architecture, probe placement, and data selection. To date, few research has demonstrated an embedded IDS within a real satellite system. This paper addresses that gap by presenting design strategies and an implementation of such an IDS on a space system simulator developed as part of the CSS (Cybersecurity for Space Systems) Project, based on NASA's NOS³ platform.

Due to the current lack of satellite-focused datasets, and considering the aforementioned constraints, we chose to favor heuristics-based detection in this article. Indeed, this allows for precise and explainable results based only on the attack's principle.

To summarize, this paper proposes three main contributions:

- The description of an embedded IDS designed for space systems. It employs a multi-probe, multi-module approach and is designed to cover a broad range of potential attack scenarios.
- A first proof of concept of our IDS implemented on a satellite simulator based on NASA's NOS³ platform¹.
- The design and implementation of a set of attack scenarios in a dedicated attack module that allows us to assess the relevance of our IDS.

This paper is organized as follows. The Section II describes related research work. Section III describes the context of the project in which this research work is done and presents the satellite simulator built within the project, based on the NOS³ platform. Section IV describes the envisaged threat model and the different attack scenarios considered and implemented in

This work was supported by IRT Saint-Exupéry as part of the CSS project.

¹Available at : <https://gitlab.laas.fr/lolive/edcc-sat-ids>

the simulator. Section V presents our IDS, the probes and the detection strategies considered. Section VI gives implementation details on the corresponding proof of concept. Section VII describes the experiments that we carried out to assess the relevance of our IDS and evaluates the efficiency of the detection. Section VIII proposes a discussion about the limits of the approach. Section IX concludes this paper and describes some perspectives to this research work.

II. RELATED WORK

To date, only a few published works study embedded Intrusion Detection Systems (IDS) in satellites, and are mostly theoretical. In [5], foundational IDS concepts for CubeSats are discussed, but not implemented in practice. A follow-up study [6] proposes a CAN bus-based IDS using machine learning, combining time and content-based detection modules, though the experimental setup used is not satellite-specific. In [4], the architecture and vulnerabilities of the NOS³ simulator [7] are analyzed, and IDS concepts are explored, though without implementation. Finally, [8] introduces a concept of multi-probe, multi-solution IDS architecture, while emphasizing the need for explainable machine learning to build trust in autonomous detection systems.

Several IDSs for space systems operate on the ground segment, performing packet inspection [9], [10]. For example, [9] proposes a hybrid deep learning-based IDS combining signature and anomaly-based detection, while [10] compares four hybrid machine learning and deep learning algorithms. Although ground-based detection is relevant due to satellite-ground communication, it has limitations: as it only analyzes data received at the ground, it may miss onboard malware, especially during communication outages. Additionally, many of these approaches focus on satellite-ground network security and Software-Defined Networking (SDN), relying on network data, whereas on-board system data could also enable effective intrusion detection.

In related embedded system domains like automotive and IoT, two main intrusion detection strategies prevail: Machine Learning (especially Deep Learning) to model normal behavior and detect anomalies, and non-AI techniques offering explainable detection tailored to specific systems. Deep Learning methods [11]–[13] effectively analyze communication data but depend heavily on large, high-quality datasets, which are lacking for satellites. Moreover, Deep Learning models often function as opaque black boxes and demand substantial computational resources, making them challenging to deploy in resource-constrained satellite environments.

In contrast, non-AI approaches, such as those in [14]–[17], are also explored. These include Finite-State Automata [14], Packet time-interval heuristics [15], Transition Matrix on ID sequences [16], and histogram-based detection [17]. These methods offer the advantage of being highly explainable but require system-specific design, necessitating a deep understanding of the target system’s operation.

In summary, while promising, these approaches often lack experimental results or fail to provide satellite-specific IDS

solutions. Additionally, when they do, the communication bus used is not space-specific. In our work, we choose to consider the NOS³ Software Bus architecture, a space-specific and widely used bus designed for its genericity and reusability. While Machine Learning approaches can yield good detection results, we prefer non-AI approaches when possible due to the absence of satellite-specific data and for better explainability.

III. SATELLITE SIMULATOR

The research work presented in this paper is carried out in the context of the *Cybersecurity for Space Systems* (CSS) Project, led by the French Institute for Technological Research (IRT) Saint Exupéry in Toulouse. One of the goals of this project is to build a state-of-the-art Space System Simulation platform for Cybersecurity Research and Development, where various advanced techniques for attack and defense can be developed and evaluated.

Several proprietary and open-source satellite simulators exist, including ESA MO Nanosat Framework [18], ESA SIMULUS [19], OpenSatKit [20], Ansys STK [21], and NASA NOS³ [7]. Academically, custom simulators are often preferred [22]. NASA NOS³ stands out as a widely-used open-source platform, offering an integrated virtual machine with Flight Software, Ground Software, and visualization tools. Its selection for developing a CubeSat constellation simulator is due to its open-source nature, implemented communication protocol stack, proven relevance in cybersecurity research [23], [24], and operational use in NASA missions [25], [26]. The custom fork used (v1.6.2) includes three components: the COSMOS ground station simulator, the visualization and sensor data module (42), and the satellite simulator based on NASA’s core Flight Software (cFS).

This version uses Consultative Committee for Space Data Systems (CCSDS) [27] communication protocols, enabling Telecommands (TC) for satellite control and Telemeasures (TM) to receive data. A new Inter-Satellite Link (ISL) component was added to the satellite Flight Software to support multi-spacecraft configurations by routing messages within the constellation. When a message arrives from the ground, ISL checks if it’s addressed to the satellite; if so, it forwards the packet to the Command Ingest (CI) component, which strips TC encapsulation and sends it to the Software Bus (SB). The SB, using CCSDS’s Space Packet Protocol (SPP), distributes the message to subscribed components, which then act or respond via TM. Figure 1 illustrates this transmission chain.

IV. ATTACK SCENARIOS

This section describes the envisaged threat model and the main attack scenarios considered and currently implemented in the simulator. This was supported by a risk analysis (EBIOS-RM method) carried out in the CSS project.

A. Threat Model

Satellites mainly communicate with ground stations, so attackers often target the ground segment as the simplest way to compromise a satellite network. Several studies examine this

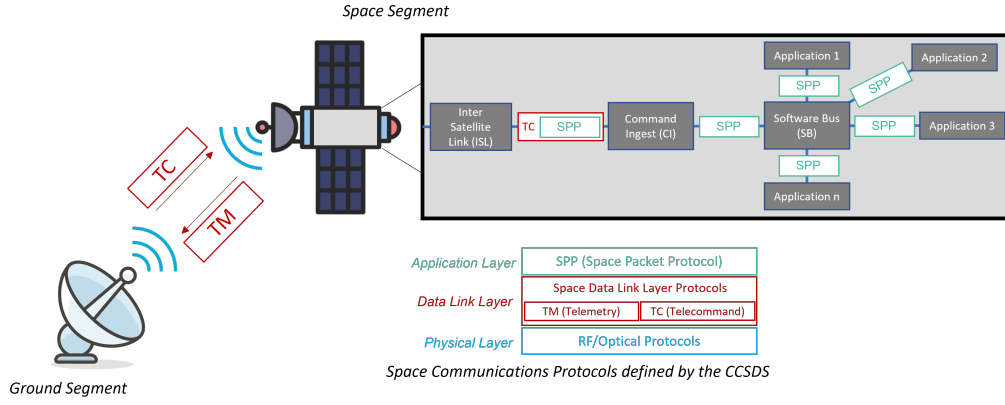


Fig. 1. Transmission chain of the simulated satellite

vector, for example, a compromised ground station sending illegitimate commands to the satellite [28]).

Attackers could try to use a “rogue” antenna to send commands, but this requires precise tracking during limited visibility and is made much harder if the satellite-ground link uses cryptography; therefore compromising a legitimate ground station remains the simplest attack path.

Another significant threat is satellite software compromise, especially the *Payload*, via supply chain vulnerabilities before launch. The risk is heightened by the diversity of suppliers required for satellite manufacturing. Additionally, as software updates are often done over-the-air, the *Payload* could be compromised during operation. Such attacks may go undetected by traditional IDS systems monitoring the satellite-ground link, leading to the need for onboard IDS. In this paper, we assume that the flight system software is trusted, but compromised *Payload* software could initiate malicious actions at any time. Thus, the study focuses on two main threats: ground segment compromise and *Payload* compromise.

B. Attacks Overview

An attacker able to take control of a ground station can craft as many TCs as wanted, invalid TCs (with invalid SPP header identifiers) or valid TCs with purposely crafted parameters. A compromised *Payload* can damage satellite integrity (e.g., corrupt flight components) or perform covert/targeted actions such as leaking confidential data or altering *Payload* outputs before transmission. For instance, an observation satellite could send falsified images or remove specific parts of photos. Some examples implemented in our platform are provided in the next Section. Whether the attacks are carried out from the ground or from the *Payload*, two kinds are considered.

A single TC can be enough to cause catastrophic damage. Examples include a TC that turns off onboard calculators, which may be legitimate for rebooting but can be abused to disrupt the mission, or a TC that forces the satellite into a fail-safe configuration at an inappropriate time.

Complex attacks can be carried out using a sequence of individually benign TCs that, when combined, trigger malicious behavior. Examples include sending multiple TCs to stealthily

TABLE I
ATTACKS SUMMARY

Attack ID	Short description	Source	Target
FA ₁	Network flooding	Ground	Any
FA ₂	Network flooding	<i>Payload</i>	Any
CA ₁	App Kill	Ground	CI, ISL
CA ₂	App Kill	<i>Payload</i>	CI, ISL
AD ₁	App Erase	Ground	Any
AD ₂	App Erase	<i>Payload</i>	Any
SW ₁	Files Wipe	Ground	Any
SW ₂	Files Wipe	<i>Payload</i>	Any
SP ₁	Satellite Spin	Ground	RW
SP ₂	Satellite Spin	<i>Payload</i>	RW
CR ₁	System Crash	Ground	CAM
CR ₂	System Crash	<i>Payload</i>	CAM
CR ₃	System Crash	Ground	Cryptolib
HJ ₁	Hijacking	Ground	Cryptolib

alter the satellite’s trajectory or a sequence that uploads a malicious file to replace a satellite component.

C. Detailed Attacks Description

This section describes the specific attacks implemented in the simulation platform. Some are real-world attacks from the literature, others are designed for the CSS project. Attacks can be launched from both the ground segment (odd numbering) or a malicious *Payload* (even numbering), unless specified. Table I summarizes the attacks. Most attacks are Denial-of-Service (DoS); the exception is **HJ**₁, which targets full takeover of the satellite.

1) *Flooding attack (FA)*: This attack aims to overload the satellite communication bus by flooding it with TCs.

2) *Cyber-ASAT [4] attacks (CA)*: In [4] the attacker triggers a DoS by terminating the CI component (the entry point within NOS³ that forwards incoming packets to the SB) via a crafted TC, which blocks communication with the satellite’s internal components. Any element in the transmission chain could be targeted to produce the same effect; we also consider a variation that targets the ISL component.

3) *App Deletion attack (AD)*: The attack aims to cause a DoS on a satellite component by deleting its binary (disk image, e.g., a .so file) and terminating its running RAM image

so operators cannot restart it. It is executed with two TCs: one to remove the file and a second to stop the application.

4) *System files Wipe attack (SW)*: This attack erases critical system files in the `/cf` directory of NOS³, including configuration files (.txt, .tbl) and application libraries (.so), the latter preventing affected applications from being restarted. It is executed with a single TC.

5) *Satellite Spin attack (SP)*: This attack aims to sabotage the mission by inducing rapid rotation about all three axes (X, Y, Z), which would, for example, prevent accurate Earth imaging. It is executed with four TCs: first disable attitude control, then send a rotation-start command for each axis using the reaction wheels.

6) *Flight Software crash attacks (CR)*: Some attacks exploit software bugs in NOS³. We found a vulnerability that causes the Flight Software (cFS) to crash and reboot when the camera (default *Payload* in NOS³) is taking a picture and is immediately restarted via a cFS command; the resulting segmentation fault can be used to produce a repeated Denial-of-Service. A second vulnerability exists in the NOS³ Cryptolib (CCSDS SDLS-EP implementation) that contains out-of-bounds vulnerabilities [29], [30]; a malicious TC with a Security Parameter Index (SPI) larger than the source-code maximum crashes Cryptolib (and ultimately the Flight Software). This Cryptolib exploit is triggered only from the ground (CR₃).

7) *Satellite hijacking attack (HJ)*: A community-discovered vulnerability in the NOS³ Cryptolib lets an attacker bypass satellite encryption by using specific SPI values that exploit uninitialized Security Associations [31]. With this capability an attacker can send unencrypted TCs, then modify the Security Associations (including keys) to lock out legitimate ground operators and hijack the satellite. This attack is executed only from the ground segment (HJ₁).

D. Attack Automation

Another contribution of this work is the design and the implementation of a generic `Input Generator` module (see Section VI-B) for the NOS³ platform that drives both legitimate mission scenarios and malicious command sequences. It is used to schedule and execute attack scenarios in simulations, so that the effectiveness of detection strategies can be assessed.

V. EMBEDDED INTRUSION DETECTION SYSTEM

A. Probe deployment

Detecting the different attack scenarios described in the threat model requires the deployment of specific probes at adapted locations onboard. Let us note that the choice of these locations is crucial from a security standpoint, to detect intrusions as early as possible. The detection strategies discussed in next section follow the Defense In-Depth and Least Privilege security design principles of the ECSS-E-ST-80C standard [32].

Three probes were designed and implemented to feed detection strategies. **Probe 1** (ISL arrival) collects TCs arriving

at the ISL component to capture network-level features (bandwidth, inter-packet times, packet size, header characteristics, satellite identity). Because cryptography is assumed enabled, it cannot read packet payloads, only flow-level metadata. **Probe 2** (CI, pre-Software Bus) is located in the CI component, analyzing deciphered TCs coming from the ground before they reach the Software Bus (avoiding bus-internal noise). It captures command types and parameters. **Probe 3** (Software Bus) is implemented inside the Software Bus to observe internal exchanges between satellite components; it captures the same information as **Probe 2** as well as the sender identity, which enables component-level filtering and is especially useful to detect malicious behaviour from a compromised *Payload*. Data from the three probes, flow metadata (from **Probe 1**), deciphered command types/parameters (**Probe 2**), and internal message data plus sender IDs (**Probe 3**), serve as input for the intrusion detection strategies. The system architecture and probe placements appear in Figure 2, and the specific detection methods are described in the next Section.

B. Intrusion detection strategies

1) *Using Probe 1 information*: Several detection mechanisms are implemented. **Flooding detection** monitors inter-packet times to flag packets sent too quickly, detecting attacks like Denial of Service (DoS) flooding from the ground (e.g., FA₁). **CRC verification** checks the Cyclic Redundancy Check of each TC; failing packets trigger alerts and are dropped, useful against jamming or malformed TC attacks. **Cryptolib vulnerability detection** analyses TC headers for invalid values that cause cFS crashes (CR₃) or encryption bypass (HJ₁). The latter requires knowledge of initialized Security Associations (SA) to monitor and enforce controls. While this attack is specific to NOS³, monitoring the SAs can still be interesting on other satellite systems. **Packet size verification** checks the size of the packets. Since encrypted communications use block ciphers with fixed block sizes, any packet length not a multiple of the block size is flagged as suspicious.

2) *Using Probe 2 information*: A simple same-packet anti-flooding was implemented to prevent same consecutive commands bypassing the flooding detection of **Probe 1** to go through. Indeed, it seems suspicious and unlikely to have quick occurrences of the same command in a real mission scenario.

3) *Using Probe 2 or Probe 3 information*: A *TC analyzer* was implemented to validate SPP packets and enforce a blacklist of dangerous TCs. It targets single-TC attacks (CA_{1,2}, SW_{1,2}) and multi-TC attack sequences (AD_{1,2}, SP_{1,2}, CR_{1,2}), whether those commands come from the ground, or from the Software Bus (in the case of a compromised *Payload*).

4) *Using Probe 3 information*: An *Application Task Profiling* detection strategy monitors messages on the Software Bus to identify anomalies based on models of legitimate message sets for each application during normal operation. This helps detect malicious activity from a compromised *Payload* trying to send unauthorized messages to other applications (AD₂, CA₂, SW₂, CR₂). Additionally, an AI-based detection

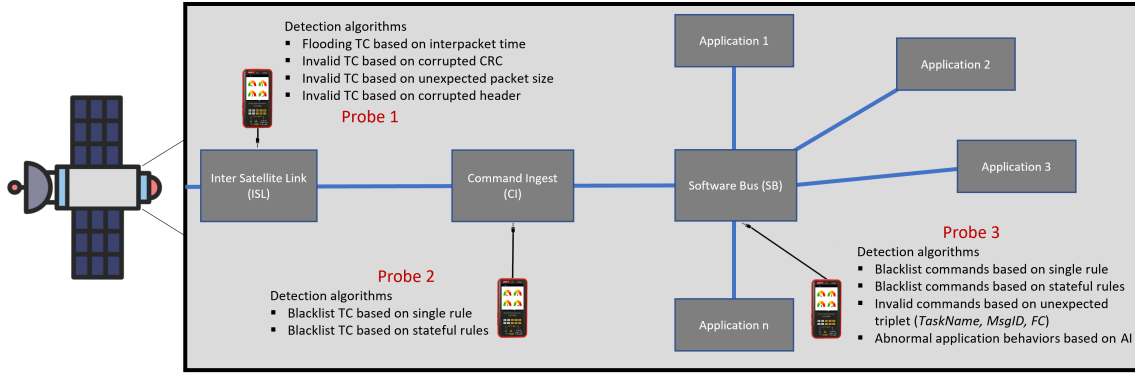


Fig. 2. Embedded probes in NOS³ architecture

strategy is being developed to spot dangerous patterns in TCs correlated with their timestamps. Because all satellite applications communicate via the Software Bus, this approach can monitor both ground-sent and internal messages. The main challenge is building an accurate model of normal message exchanges that fits within the satellite's limited memory and computational resources.

Let us note that all alerts must be transmitted to the ground, as no human presence is assumed onboard to analyze them. These alerts are sent via specific telemetry commands. A prioritization mechanism can be considered, ensuring that telemetry commands containing security alerts are transmitted with the highest priority.

C. Detection versus prevention

While the paper focuses on intrusion detection, the designed probes and detection algorithms can also support intrusion prevention. By switching from raising alerts to actively dropping suspect TCs, the system functions as an Intrusion Prevention System (IPS). The probe placements and detection logic remain unchanged; only the response varies depending on the satellite's operational context. When communication with the ground is available, the system raises alerts to let human operators investigate and respond. Otherwise, alerts are replaced by automated prevention (dropping suspect TCs). For critical attacks (e.g., CA_{1,2}), which can block further command reception, detection modules are configured to always drop such dangerous TCs.

VI. IMPLEMENTATION DETAILS

This section is dedicated to the detailed presentation of the probes and detection strategies of our IDS proof of concept currently implemented in the NOS³ platform.

A. Probes and detection algorithms

1) *Probe 1*: A probe located at the satellite's entry point (within the ISL component) monitors incoming packets at the reception level before decryption. Since the payload is encrypted (assuming cryptography is enabled), detection is based on metadata and header analysis. The following metrics are used to detect anomalies.

a) *Inter-packet times*: A flooding detection mechanism, using a window of 10 packets, flags new packets when the average of last intervals is below 20 ms, based on observations using the COSMOS tool, where normal command spacing ranges from 10–50 ms. In operational contexts, added delays for telemetry confirmation further reduce the likelihood of legitimate high-frequency commands.

b) *CRC field validity*: If the CRC is invalid, the packet is dropped. While this can result from transmission errors (not necessarily malicious), regular reporting of dropped packets to the ground is considered valuable for situational awareness.

c) *Packet size*: Under encrypted communications (using AES-CBC-256), packets should have lengths that are multiples of 256 bits. Any packet with a size not matching this constraint is flagged as suspicious.

d) *TC header values*: While the payload is encrypted, TC headers remain in plaintext and are inspected for specific attacks:

- **CR₃**: the SPI field is checked to ensure it does not exceed the maximum number of defined Security Associations (set to 64). An overflow here can crash the Cryptolib and subsequently the cFS.
- **HJ₁**: detection involves verifying that the SPI used is part of the pre-configured, initialized Security Associations. Unauthorized SPIs may indicate attempts to bypass encryption and hijack the satellite.

These checks together allow the ISL-level probe to detect a variety of low-level attacks even with encryption.

2) *Probe 2*: The second probe is located within the CI component, immediately after incoming packets are decrypted. At this stage, the system has access to the full SPP packet contained in the TC payload. A *TC analyzer* is implemented here to identify a malicious activity by applying a set of static rules that flag blacklisted commands. Each packet is identified using a combination of its *MsgID* and Function Code *FC*. The *MsgID* distinguishes between TCs and TMs and helps determine the destination or source application, since only one application subscribes to or emits a given message type. The *FC* further specifies the sub-command in the case of TCs. Rules are defined using this *MsgID* and *FC* pair, are parsed

at the start of the simulation, and stored in RAM for efficient access.

Both standalone and stateful rules can be defined. Single rules simply identify a specific (MsgId, FC) couple inside one single packet considered as malicious. Stateful rules correspond to the detection of the combination of several packets, that together, are considered as malicious. These attack scenarios are represented by state machines, the transition between states corresponding to the reception of one of the malicious packets that compose the attack scenario. This principle is very similar to the state machines that are used in the IP packet filters, and is crucial to be able to detect more advanced attack scenarios, composed of a sequence of different packets.

We also implemented a same-packet anti-flooding, aiming at preventing consecutive identical commands escaping P1's anti-flooding to go through. It relies on two empirically determined thresholds, an inter-packet time (set to 40ms) and a number of consecutive identical packets (set to 4). In other words, 4 packets with an inter-packet time under 40ms, sharing the same couple (MsgId, FC), are flagged as "soft-flooding". The combination of the anti-flooding mechanisms of **Probe 1** and **Probe 2** allows to drastically limit the effects of flooding.

3) *Probe 3*: The Bus probe is embedded within the SB component. This placement enables the interception of messages sent by any application. The probe logs SPP packets along with relevant metadata, including timestamps, TaskName (emitting application task), MsgId, and FC, as well as the whole packet's payload, for subsequent analysis by detection algorithms. In the same way as for **Probe 2**, a *TC analyzer* is implemented allowing the definition of rules, identifying blacklisted commands.

An additional detection layer, called the *Task Profiling* module, is designed to detect anomalies in messages transmitted over the Software Bus by modeling the legitimate message set for each application task during normal operations. This model is built by monitoring all messages exchanged during a complete orbital pass simulation without any attacks, resulting in a structured set of triplets in the form of (TaskName, MsgID, FC). Using this profile, the detection system compares real-time traffic to expected behavior, enabling it to identify deviations that may suggest malicious activity. Consequently, this layer can generate alerts or drop packets, depending on the current strategy, when an application attempts to communicate with another in an unauthorized manner.

This detection layer, however, cannot identify cases where an application impersonates a legitimate one but slowly deviates from its normal behavior. Therefore, an AI-based detection module is under development. Its aim is to build a model of legitimate bus exchanges by analyzing metadata of packets captured by the **Probe 3**, to perform anomaly detection. An automated pipeline is being created to introduce randomness in attack timing during simulations, to diversify the dataset and improve model generalization. Algorithms like XGBoost and Isolation Forest are being explored due to their favorable balance of performance, embeddability, and interpretability.

B. Input Generator

A specific tool, so-called Input Generator, used for command automation and scenario design, has been developed. It is written in Python and composed of several modules allowing to send fully customized SPP commands using the correct syntax of the NOS³ environment. Command syntax is parsed from the COSMOS configuration files, and the selected command (and its attributes) is stored as a JSON file. This allows for an easy editing of the parameters either by-hand or by scripting. Commands can then be sent to the satellite.

VII. EXPERIMENTS

A. Attack detection performances

We conducted experiments involving both legitimate and malicious traffic. The legitimate traffic corresponds to a simple satellite mission, during which the *Payload*, a camera, is tasked with capturing images periodically and transmitting them to the ground via telemetry. We first evaluated the **Probe 1** and **Probe 2** detection strategies. 1000 attacks were picked randomly and sent with a 5 second interval (*Exp 1*). All attacks were successfully detected, with no false positives. We also assessed the accuracy of the anti-flooding, by sending over 1000 packets at once without delay (*Exp 2*). Except for the very first packets (corresponding to the filling of the sliding window used for the detection) which were not detected, all the packets constituting the flooding attack were successfully detected as flooding. When the anti-flooding mechanism of **Probe 2** is used at the same time, we reduce the initial non-detection to the four first packets of each flooding attack (according to the threshold set to 4 packets). However, these anti-flooding mechanisms may also generate false positives when some legitimate consecutive TCs are sent quickly (for example with an onboard library update, which may generate a lot of file data transfer TCs). Our experiments show that with the anti-flooding mechanism of **Probe 2** enabled, transfers of files exceeding 3KB generate too many packets and are blocked. When only the **Probe 1** anti-flooding is used, files up to 4KB can be transferred. In comparison, the size of a satellite component library (.so) in our environment is typically of 100KB. CFDP (CCSDS File Delivery Protocol) supports two transfer modes, either UDP-like or TCP-like. In real missions, the latter will likely be preferred to ensure correct file transmission. In this case, the anti-flooding mechanism will only result in delays due to dropped packets being retransmitted. For missions using the unreliable transfer mode, our implementation allows to disable the anti-flooding on file transfers, although this is not recommended.

Probe 3 was also evaluated, with the *Task Profiling* and the *TC Analyzer* being tested separately in a scenario using a malicious *Payload*. The *Payload* was set to send every onboard attack presented in IV-C, except for the FA₂ attack (which will be handled by the future AI module). A total of 1000 attacks were randomly picked among the 6 available. We considered two different experiments: 1) attacks are sent without any delay between them (*Exp 4*), and 2) attacks are

sent with a 10 second interval, with an inter-command delay (single-command attacks included) randomized between 1-200 ms (*Exp 5*). The *TC Analyzer* and *Task Profiling* modules correctly detected all the attacks with no false positive or false negative, which confirms the relevance of the stateless and stateful rules we have implemented. Attack packets were tagged as malicious to evaluate packet-by-packet the correctness of the *TC Analyzer*'s output.

B. Code footprint, latency overhead, CPU and memory usage

Experiments were also carried out to analyze the impact of the IDS's security controls on satellite performance, focusing specifically on code footprint, latency overhead, and CPU and memory usage. The satellite simulator runs on a server equipped with an AMD EPYC 7763 processor running at 2.6GHz. The virtual machine running the space segment (*core-cpul* process) uses 6 cores and 8GB of RAM.

The code footprint was measured by calculating the percentage of lines of code added to the simulator to implement our security controls, relative to the total number of lines of code (IDS included), in this case 1682 over 281209 total (roughly 0.6%).

The experiments below have all been conducted through three different setups : (1) without the IDS enabled, (2) with the IDS enabled and the default rule set of the *TC analyzer* - approximately 20 rules - or (3) with a set of 1000 rules, to evaluate the overhead a very large amount of rules could induce.

The latency overhead experiments were carried out over 300 messages, each sent with a 1 second interval to ensure non-overlapping. We measured the overhead of the processing of each message in the satellite simulator, both in simulated and real time. The simulated time was not accurate enough (1 ms steps), preventing a correct measurement of the latency overhead. Even if the real time can vary according to the load on the server running the virtual machine, using the *timespec_get* function, we could identify an overhead of around 21 μ s for the default ruleset and 34 μ s with 1000 rules. It corresponds to an increase of 8.7% and 14%, respectively. This additional time can seem substantial, but in comparison to the message rate on the SB (100 messages per second typically, equivalent to approximately 1 message every 10ms) it is reasonable enough not to have a significant impact on message processing.

CPU usage was measured using the Linux utility *ps* on the whole space segment process. As shown in Table II, no difference in CPU load was observed during simulation runs, regardless of whether the IDS was enabled. This indicates that the CPU overhead introduced by the security checks is negligible. For memory usage, we focused on measuring the stack size used by the simulator. Since our detection modules implement security controls without dynamic heap allocation (relying solely on local variables stored on the stack), we used two Linux utilities to assess stack memory: the *pmap* utility and the *VmStk* field in the */proc/[pid]/status* file. Over one hour of simulation, no difference in the number

of allocated stack pages was detected whether the IDS was enabled or not. We then manually calculated the size of the local variables in our IDS source code to estimate the overhead introduced in the stack size. We obtained a result of approximately 374 bytes (worst case scenario with parallelism). In comparison with the default size of the stack memory allocated (132KB), this overhead is negligible. However, this does not take into account the internal structures of each probe, or the rules declared in the *TC analyzer*, which end up in the *.bss* segment of the program. With the default *TC analyzer* ruleset, we obtain a *.bss* impact of 1070 bytes, and with 1000 rules we obtain 132270 bytes respectively. As the *.bss* memory requirements scale with the number of rules, our current ruleset shows a low impact, which is not the case when using 1000 rules, where the impact on the memory is significant. However, this memory increase does not hinder the correct behavior of the satellite. Indeed, all the packets are still correctly processed without any loss, even if a small latency is induced. Moreover, this 1000 rules setup is a pessimistic setting of the IDS, and seems highly unlikely to be used in such a resource-constrained environment. As summarized in Table II, these results demonstrate that our IDS implementation is fully compatible with the strict resource constraints of space environments, given that the number of rules used is realistic.

TABLE II
PERFORMANCE IMPACT OF THE IDS COMPARED TO BASELINE

Metric	IDS ON - Default	IDS ON - 1000 rules
CPU usage	non-significant impact	
Memory usage (stack)	+374B	
Memory usage (BSS)	+1070B	+132270B
Latency	+21 μ s (8.7%)	+34 μ s (14%)

VIII. DISCUSSION

The IDS was designed with the NOS³ platform in mind, as all experiments were conducted on it. However, the architecture is considered generic and adaptable to other embedded space systems. The probe placements target strategic points in the system (e.g., at TC arrival, after decryption, on the communication bus) based on the threat model, specifically, attacks from a malicious ground station or compromised *Payload*, not on NOS³-specific features. Moreover, the communication protocols used (CCSDS TC and SPP) are widely adopted across the space industry, being used in over 1000 missions according to the CCSDS website [33]. This supports the broad applicability of the proposed detection strategies.

A whitelist approach, from a security standpoint, should be preferred in the *TC Analyzer* to provide an improved coverage. However, using this approach requires deep expert knowledge of the satellite's operation and mission, which can prove to be difficult. In our case, it was more practical to define known attack patterns than to enumerate all legitimate commands. The coverage difference between these approaches results in gaps in the defense. We intend to reduce them with the future anomaly detection module.

The **Probe 3** is currently implemented in a cFS API used to broadcast on the SB. This is a current limit because this probe can be bypassed by an attacker if a malicious *Payload* implements its own broadcasting function, directly calling the underlying OS queue API to send packets. Instrumenting this queue implementation with a new probe would be the most effective way to keep track of every packet.

Another limitation of this work is the assumption that the cFS is inherently trusted and that only the *Payload* may be compromised. In reality, the cFS itself could be corrupted during updates throughout the satellite's lifetime. Since **Probe 3** is implemented within the cFS, an attacker could potentially remove it from the software bus code, disabling its detection functionality. In such a scenario, a fallback probe at the OS level could preserve some detection capabilities, though in a degraded mode. It's important to note that in the context of NOS³, a fully software-based platform for small satellites like CubeSats, trust must inherently be placed in software components for detection. In contrast, more advanced satellites with dedicated onboard hardware buses could employ both hardware and software probes, removing the need to rely on software integrity. However, this would not impact the core IDS design, as the detection approach, based on monitoring communications on the bus and the nature of the data collected, remains the same regardless of whether the probe is implemented in software or hardware.

IX. CONCLUSION

The paper presents the design and implementation of an Intrusion Detection System (IDS) for satellites. It outlines the threat model, attack scenarios, and the IDS architecture. A proof of concept was developed on the NOS³ platform, along with software to simulate attacks. Experiments show that the IDS effectively detects intrusions without disrupting satellite operations.

The near-future work focuses on completing the implementation of several components presented in the paper. This includes adding a fuzzing capability to the Input Generator to allow for enhanced testing, and developing an AI behavioral module based on **Probe 3**, along with a dataset for its training and testing. Additionally, the research explores refining detection strategies, such as extending packet size verification to unencrypted communications and improving CRC verification using new metrics like the frequency and percentage of packets with bad CRCs over time. These efforts aim to strengthen the intrusion detection system's effectiveness.

REFERENCES

- [1] "Case study: Viasat attack," WebPage, Jun. 2022. [Online]. Available: <https://cyberconflicts.cyberpeaceinstitute.org/law-and-policy/cases/viasat>
- [2] "Thales seizes control of esa demonstration satellite in first cybersecurity exercise of its kind," WebPage, Apr. 2023. [Online]. Available: https://www.thalesgroup.com/en/worldwide/security/press_release/thales-seizes-control-esa-demonstration-satellite-first
- [3] M. Werremeyer, J. Williams, S. Wood, M. Walker, J. Ameen, and B. Kerley, "Hack-A-Sat: Four Years from the Cromulence Tech Team," in *2024 IEEE Aerospace Conference*, Mar. 2024, pp. 1–17, iSSN: 1095-323X. [Online]. Available: <https://ieeexplore.ieee.org/document/10521107>
- [4] A. Schalk and D. Brown, "Detection and Mitigation of Vulnerabilities in Space Network Software Bus Architectures," in *2023 IEEE Aerospace Conference*, Mar. 2023, pp. 1–10, iSSN: 1095-323X. [Online]. Available: <https://ieeexplore.ieee.org/document/10115986>
- [5] O. Driouch, S. Bah, and Z. Guennoun, "Intrusion detection system for CubeSats: a survey," in *2023 International Wireless Communications and Mobile Computing (IWCMC)*, Jun. 2023, pp. 596–601, iSSN: 2376-6506. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10182729>
- [6] —, "CANSat-IDS: An adaptive distributed Intrusion Detection System for satellites, based on combined classification of CAN traffic," *Computers & Security*, vol. 146, p. 104033, Nov. 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404824003389>
- [7] "Nasa operational simulation for small satellites," WebPage. [Online]. Available: <https://www.nasa.gov/nasa-operational-simulation-for-small-satellites/>
- [8] S. Gios, C.-H. Bertrand Van Ouytsel, M. D. Caribé, and A. Legay, "A vision on a methodology for the application of an Intrusion Detection System for satellites," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. Sacramento CA USA: ACM, Oct. 2024, pp. 2205–2209. [Online]. Available: <https://dl.acm.org/doi/10.1145/3691620.3695314>
- [9] U. Uhongora, R. Mulinde, Y. W. Law, and J. Slay, "Deep-learning-based Intrusion Detection for Software-defined Networking Space Systems," *European Conference on Cyber Warfare and Security*, vol. 22, no. 1, pp. 639–647, Jun. 2023, number: 1. [Online]. Available: <https://papers.academic-conferences.org/index.php/eccwsw/article/view/1085>
- [10] A. T. Azar, E. Shehab, A. M. Mattar, I. A. Hameed, and S. A. Elsaid, "Deep Learning Based Hybrid Intrusion Detection Systems to Protect Satellite Networks," *Journal of Network and Systems Management*, vol. 31, no. 4, p. 82, Sep. 2023. [Online]. Available: <https://doi.org/10.1007/s10922-023-09767-8>
- [11] M.-J. Kang and J.-W. Kang, "A Novel Intrusion Detection Method Using Deep Neural Network for In-Vehicle Network Security," in *2016 IEEE 83rd Vehicular Technology Conference (VTC Spring)*, May 2016, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/7504089>
- [12] S. Khandelwal, E. Wadhwa, and S. Shreejith, "Deep Learning-based Embedded Intrusion Detection System for Automotive CAN," in *2022 IEEE 33rd International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, Jul. 2022, pp. 88–92, iSSN: 2160-052X. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9912045>
- [13] M. Ge, X. Fu, N. Syed, Z. Baig, G. Teo, and A. Robles-Kelly, "Deep Learning-Based Intrusion Detection for IoT Networks," in *2019 IEEE 24th Pacific Rim International Symposium on Dependable Computing (PRDC)*, Dec. 2019, pp. 256–25609, iSSN: 2473-3105. [Online]. Available: <https://ieeexplore.ieee.org/document/8952154>
- [14] Y. Fu, Z. Yan, J. Cao, O. Koné, and X. Cao, "An Automata Based Intrusion Detection Method for Internet of Things," *Mobile Information Systems*, vol. 2017, no. 1, p. 1750637, 2017, eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1155/2017/1750637>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2017/1750637>
- [15] H. M. Song, H. R. Kim, and H. K. Kim, "Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network," in *2016 International Conference on Information Networking (ICOIN)*, Jan. 2016, pp. 63–68. [Online]. Available: <https://ieeexplore.ieee.org/document/7427089>
- [16] M. Marchetti and D. Stabili, "Anomaly detection of can bus messages through analysis of id sequences," in *2017 IEEE Intelligent Vehicles Symposium (IV)*, 2017, pp. 1577–1583.
- [17] S. J. J. Génereux, A. K. H. Lai, C. O. Fowles, V. R. Roberge, G. P. M. Vigeant, and J. R. Paquet, "MAIDENS: MIL-STD-1553 Anomaly-Based Intrusion Detection System Using Time-Based Histogram Comparison," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 56, no. 1, pp. 276–284, Feb. 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/8705276>

- [18] ESA, “NanoSat MO Framework,” 2025, <https://nanosat-mo-framework.github.io/>.
- [19] —, “SIMULUS-NG: a new era for satellite simulation,” 2024, <https://esoc.esa.int/content/simulus-ng-new-era-satellite-simulation>.
- [20] , “OpenSatKit : Core Flight System (cFS) Application Developer’s Kit,” 2023, <https://github.com/OpenSatKit/OpenSatKit>.
- [21] Ansys, “Ansys STK : Software for Digital Mission Engineering and Systems Analysis,” 2025, <https://www.ansys.com/fr-fr/products/missions/ansys-stk>.
- [22] Y. Zhang, Q. Wu, Z. Lai, Y. Deng, H. Li, Y. Li, and J. Liu, “Energy drain attack in satellite internet constellations,” in *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*. IEEE, 2023, pp. 1–10.
- [23] B. Bailey, “Nasa iv&v’s cyber range for space systems,” in *Annual Ground System Architectures Workshop*, ser. GSAW, no. GSFC-E-DAA-TN65725, 2019.
- [24] Hack-A-Sat, “Hack-A-Sat: Capture the Flag Competition,” 2024, <https://hackasat.com/>.
- [25] NASA, “Core Flight System (cFS),” 2024, <https://github.com/nasa/cFS>.
- [26] NASA Goddard Space Flight Center, “The Spark: Fall 2024,” 2024, https://partnerships.gsfc.nasa.gov/wp-content/uploads/The_Spark_Fall_2024_DIGITAL-S.pdf.
- [27] CCSDS, “Space Packet Protocol,” WebPage, Jun. 2020. [Online]. Available: https://ccsds.org/wp-content/uploads/gravity_forms/5-448e85c647331d9cbaf66c096458bdd5/2025/01/133x0b2e2.pdf
- [28] J. Theborge, W. Henry, and G. Falco, “Developing scenarios supporting space-based ids,” *ASCEND* 2022, Oct. 2022. [Online]. Available: <https://arc.aiaa.org/doi/10.2514/6.2022-4219>
- [29] “NASA Cryptolib,” WebPage. [Online]. Available: <https://github.com/nasa/CryptoLib>
- [30] Ayman Boulaich, and Andy Olchawa, and Milenko Starcik, “How to crash a Spacecraft – DoS through Vulnerability in NASA CryptoLib v1.3.0,” 2024, <https://visionspace.com/crashing-cryptolib/>.
- [31] Antonin Boulnois, “Exploring Vulnerabilities in the SDLS Implementation of NASA’s CryptoLib,” 2024, <https://securitybynature.fr/post/hacking-cryptolib/>.
- [32] ECSS, “Space engineering Security in space systems lifecycles, ECSS-E-ST-80C,” Jul. 2024, <https://visionspace.com/crashing-cryptolib/>.
- [33] CCSDS, “CCSDS Missions,” 2025, <https://ccsds.org/implementations/missions/>.